Residentia (PG Finder) — Full Project Flow & Architecture

Interview-Ready Deep Study Guide

---

1. WHAT IS THIS PROJECT?

Residentia is a full-stack **PG (Paying Guest) Finder** web application where:

- **Clients** browse, book, and pay for PG accommodations.
- **Owners** list and manage their PG properties.
- **Admins** oversee the entire platform — users, owners, bookings, reviews, requests.

---

2. TECH STACK

Layer	Technology
Frontend	React.js (Vite), React Router v6, Axios, CSS Modules
Backend	Spring Boot 3.x (Java 17), Spring Security, Spring Data JPA
Database	MySQL (hosted on AWS RDS)
Auth	JWT (JSON Web Token) — Stateless
File Storage	Cloudinary (image uploads)
Payment	Razorpay Gateway
Email	Spring Mail (Gmail SMTP)
API Docs	Swagger / OpenAPI 3
Logging	SLF4J + Logback + AOP-based LoggingAspect
Deployment	Backend on AWS EC2, DB on AWS RDS

---

3. HIGH-LEVEL ARCHITECTURE

```
┌──────────────────────────────────────────────────────────────────────┐
│                        BROWSER (React + Vite)                        │
│                                                                      │
│  PublicBrowse → Home/Login → Register → Dashboard (CLIENT/OWNER/ADMIN)│
│                                                                      │
│  Axios Interceptor (attaches JWT from localStorage to every request) │
└──────────────────────┬──────────────────────────────────────────────┘
                       │  HTTP REST (JSON)
                       ▼
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                Spring Boot Backend (Port 8080 / AWS EC2)                 │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │              Spring Security Filter Chain                        │   │
│   │  CORS → JwtAuthenticationFilter → AuthorizationFilter          │  │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│   ┌────────────────┐   ┌────────────────┐   ┌──────────────────────────┐   │
│   │  Controllers   │   │    Services    │   │       Repositories       │   │
│   │  (REST APIs)   │→  │  (Business     │→  │   (Spring Data JPA)      │   │
│   │                │   │   Logic)       │   │                          │   │
│   └────────────────┘   └────────────────┘   └──────────────────────────┘   │
│                                                          │              │
│   ┌──────────────────────────────────────────────────────┐│              │
│   │  Cross-Cutting: AOP LoggingAspect, GlobalException││              │
│   │  Handler, CloudinaryService, EmailService         ││              │
│   └──────────────────────────────────────────────────────┘│              │
└──────────────────────────────────────────────────────────────────────────┘
                                                           │
                                                           │
                        ┌──────────────────────────────────▼──────┐
                        │  MySQL on AWS RDS (residentia_db)        │
                        │  Tables: regular_users, pg_owners, pgs,  │
                        │  bookings, reviews, change_requests, admins│
                        └──────────────────────────────────────────┘

                        ┌──────────────────────────────────────────┐
                        │  Cloudinary CDN (Property Images)        │
                        └──────────────────────────────────────────┘

                        ┌──────────────────────────────────────────┐
                        │  Razorpay (Payment Gateway)              │
                        └──────────────────────────────────────────┘
```

---

4. DATABASE SCHEMA (Entity Relationships)

```
regular_users          pg_owners              admins
──────────────         ─────────              ──────
id (PK)                id (PK)                id (PK)
name                   name                   name
email (unique)         email (unique)         email (unique)
mobile_number          mobile_number          mobile_number
password_hash          password_hash          password_hash
is_active              business_name          permissions (JSON)
created_at             address                department
updated_at             verification_status    is_active
                       is_active              created_by (FK → admins)
                       created_at             last_login_at

pgs (Property)
```

```
──────────────
id (PK)
owner_id (FK → pg_owners)
property_name
address, city, state, pincode
rent_amount
sharing_type         ← (single/double/triple)
max_capacity
available_beds
food_included
description
status               ← ACTIVE / INACTIVE / PENDING
map_link
image_url (Cloudinary URL)
amenities
reviews (count)

bookings
────────
id (PK)
pg_id (FK → pgs)
tenant_name, tenant_email, tenant_phone
booking_date
check_in_date, check_out_date
amount
notes
status               ← PENDING / CONFIRMED / CANCELLED
payment_status       ← PENDING / PAID / FAILED
razorpay_order_id
razorpay_payment_id
razorpay_signature
created_at

reviews
───────
id (PK)
property_id (FK → pgs)
user_id (FK → regular_users)
rating
review_text
is_anonymous
status               ← PENDING / APPROVED
created_at

change_requests
───────────────
id (PK)
property_id (FK → pgs)
owner_id (FK → pg_owners)
change_type          ← UPDATE / VERIFY
status               ← PENDING / APPROVED / REJECTED
change_details (JSON)
created_at
```

---

5. BACKEND PACKAGE STRUCTURE

```
com.residentia
├── ResidentialiaApplication.java       ← Spring Boot Entry Point
├── config/
│   ├── SecurityConfig.java             ← Security filter chain, CORS, rules
│   ├── JwtAuthenticationFilter.java    ← Extract + validate JWT per request
│   ├── JwtAuthenticationEntryPoint.java← Handles 401 Unauthorized response
│   ├── CloudinaryConfig.java           ← Cloudinary bean configuration
│   └── SwaggerConfig.java              ← OpenAPI/Swagger setup
├── security/
│   └── JwtTokenProvider.java           ← Generate / Validate / Parse JWT
├── controller/
│   ├── AuthController.java             ← /api/auth/* (login, register)
│   ├── ClientController.java           ← /api/client/* (browse, search)
│   ├── ClientBookingController.java    ← /api/client/bookings/*
│   ├── ClientProfileController.java    ← /api/client/profile/*
│   ├── ClientReviewController.java     ← /api/client/reviews/*
│   ├── BookingController.java          ← /api/owner/bookings/*
│   ├── OwnerController.java            ← /api/owner/* (properties, profile)
│   ├── PropertyController.java         ← /api/owner/properties/*
│   ├── PaymentController.java          ← /api/payment/* (Razorpay)
│   ├── AdminDashboardController.java   ← /api/admin/dashboard
│   ├── AdminOwnerController.java       ← /api/admin/owners/*
│   ├── AdminPgController.java          ← /api/admin/pgs/*
│   ├── AdminPgBookingController.java   ← /api/admin/bookings/*
│   ├── AdminRequestController.java     ← /api/admin/requests/*
│   ├── AdminReviewController.java      ← /api/admin/reviews/*
│   ├── AdminUserController.java        ← /api/admin/users/*
│   └── FileUploadController.java       ← /api/files/*
├── service/
│   ├── BookingService.java
│   ├── PropertyService.java
│   ├── OwnerService.java
│   ├── ClientService.java
│   ├── AdminDashboardService.java
│   ├── AdminRequestService.java
│   ├── AdminReviewService.java
│   ├── CloudinaryService.java
│   └── EmailService.java
├── entity/
│   ├── RegularUser.java
│   ├── Owner.java
│   ├── Property.java
│   ├── Booking.java
│   ├── Review.java
│   ├── Request.java
│   └── Admin.java
├── dto/
│   ├── LoginRequest.java
```

```
│   ├── AuthResponseDTO.java
│   ├── BookingDTO.java
│   ├── PropertyDTO.java
│   ├── OwnerDTO.java
│   ├── RegularUserRegistrationDTO.java
│   └── DashboardResponse.java
├── repository/
│   ├── RegularUserRepository.java
│   ├── OwnerRepository.java
│   ├── PropertyRepository.java
│   ├── BookingRepository.java
│   ├── ReviewRepository.java
│   ├── RequestRepository.java
│   └── AdminRepository.java
├── exception/
│   ├── GlobalExceptionHandler.java     ← @RestControllerAdvice
│   ├── ResourceNotFoundException.java
│   ├── DuplicateResourceException.java
│   ├── UnauthorizedException.java
│   └── ErrorResponse.java
├── aspect/
│   └── LoggingAspect.java              ← AOP — logs every controller call
└── logging/
    └── ActionLogger.java               ← Helper for structured action logging
```

---

6. FRONTEND STRUCTURE

```
src/
├── App.jsx                     ← Route definitions (React Router v6)
├── main.jsx                    ← React DOM entry
├── api/
│   ├── axios.js                ← Axios instance (base URL, interceptors)
│   ├── api.js                  ← Re-export of axios instance
│   ├── bookingApi.js           ← Booking-specific API calls
│   ├── ownerApi.js             ← Owner-specific API calls
│   └── propertyApi.js          ← Property-specific API calls
├── utils/
│   └── frontAuth.js            ← Login / Register / Logout helpers + localStorage
├── routes/
│   └── ProtectedRoute.jsx      ← Role-based route guard
├── pages/
│   ├── Landing/
│   │   ├── Home.jsx            ← Login page
│   │   └── PublicBrowse.jsx    ← Public property listing (no auth needed)
│   ├── Auth/
│   │   ├── Client/             ← ClientRegister.jsx
│   │   ├── Owner/              ← OwnerRegister.jsx
│   │   └── Admin/              ← AdminLogin.jsx
│   ├── staticPages/            ← About, Contact, Help
│   ├── ClientProfile.jsx
```

```
│   └── Payment.jsx             ← Razorpay payment flow
└── components/
    ├── ClientDashboard/
    │   ├── ClientDashboard.jsx ← Main dashboard layout
    │   ├── Listings.jsx        ← Browse properties
    │   ├── ClientPropertyView.jsx ← Single property detail
    │   ├── ClientBookingPage.jsx  ← Booking form
    │   ├── MyBookings.jsx      ← User's booking history
    │   ├── ProfileContent.jsx  ← User profile edit
    │   └── SearchSection.jsx   ← Filter/search properties
    ├── OwnerPanel/
    │   ├── Owner.jsx           ← Layout + sidenav
    │   ├── DashboardContent.jsx← Stats overview
    │   ├── MyPropertiesContent.jsx ← List of properties
    │   ├── AddPropertyContent.jsx  ← Add new PG form
    │   ├── BookingsContent.jsx ← View bookings
    │   ├── ProfileContent.jsx  ← Owner profile
    │   └── ViewProperty.jsx    ← Single property detail (owner view)
    └── AdminDashboard/
        ├── AdminDashboard.jsx  ← Layout + sidenav
        ├── DashboardContent.jsx← Platform statistics
        ├── UserContent.jsx     ← Manage regular users
        ├── OwnerContents.jsx   ← Manage owners
        ├── PropertiesContent.jsx ← Manage PGs
        ├── BookingsContent.jsx ← All bookings
        ├── RequestsContent.jsx ← Approve/reject change requests
        └── ReviewsContent.jsx  ← Moderate reviews
```

---

7. THREE USER ROLES — FULL FEATURE SET

7.1 CLIENT (Regular User)

Feature	API Endpoint
Register	`POST /api/auth/register/client`
Login	`POST /api/auth/login` (role=CLIENT)
Browse PGs publicly	`GET /api/client/properties` (no auth)
Search/filter PGs	`GET /api/client/search/**` (no auth)
View PG details	`GET /api/client/properties/{id}`
Book a PG	`POST /api/client/bookings`
View my bookings	`GET /api/client/bookings/my-bookings`
Pay for booking	`POST /api/payment/create-order/{bookingId}`
Verify payment	`POST /api/payment/verify`
Submit review	`POST /api/client/reviews`

Feature	API Endpoint
Edit profile	`PUT /api/client/profile`

7.2 OWNER (PG Owner)

Feature	API Endpoint
Register	`POST /api/auth/register/owner`
Login	`POST /api/auth/login` (role=OWNER)
Add property	`POST /api/owner/properties`
Edit property	`PUT /api/owner/properties/{id}`
View my properties	`GET /api/owner/properties`
View bookings	`GET /api/owner/bookings`
View property bookings	`GET /api/owner/property/{id}/bookings`
Update booking status	`PUT /api/owner/bookings/{id}`
Upload images	`POST /api/files/upload`
Submit change request	Handled via PropertyService.createPropertyRequest()

7.3 ADMIN (System Administrator)

Feature	API Endpoint
Login (hardcoded)	`POST /api/auth/login` (role=ADMIN)
Dashboard stats	`GET /api/admin/dashboard`
Manage users	`GET/PUT/DELETE /api/admin/users/**`
Manage owners	`GET/PUT/DELETE /api/admin/owners/**`
Manage PGs	`GET/PUT/DELETE /api/admin/pgs/**`
All bookings	`GET /api/admin/bookings`
Approve/Reject requests	`PUT /api/admin/requests/{id}`
Moderate reviews	`PUT /api/admin/reviews/{id}`

---

8. COMPLETE REQUEST LIFECYCLE (End to End)

Example: Client books a PG

```
Step 1: User clicks "Book Now" on a PG card
        → React Router navigates to /book/:id
        → ProtectedRoute checks: auth.isLoggedIn && auth.role === "CLIENT"
```

```
        → If not logged in → redirect to /

Step 2: ClientBookingPage renders
        → Pre-fills form with user info from localStorage (pg_auth)
        → User fills check-in/out dates, phone

Step 3: Form Submit
        → bookingApi.createBooking(propertyId, bookingData) called
        → axios.js interceptor: attaches Authorization: Bearer <JWT>
        → POST /api/client/bookings/{propertyId}

Step 4: Backend receives request
        → CORS filter checks Origin header
        → JwtAuthenticationFilter extracts JWT from "Authorization" header
        → JwtTokenProvider.validateToken() → true
        → Extracts email, role, userId from JWT claims
        → Sets Authentication in SecurityContextHolder
        → Sets request.setAttribute("email", email)

Step 5: Spring Security AuthorizationFilter
        → Checks: /api/client/** → .authenticated() → token present → PASS

Step 6: ClientBookingController.createBooking()
        → Reads email from request.getAttribute("email")
        → Calls BookingService.createBooking(propertyId, bookingDTO)

Step 7: BookingService
        → Fetches Property from PropertyRepository
        → Creates Booking entity, sets status=PENDING, paymentStatus=PENDING
        → bookingRepository.save(booking)
        → Returns saved Booking

Step 8: Response sent to frontend
        → JSON with bookingId returned
        → React navigates to /payment/:bookingId

Step 9: Payment Flow
        → POST /api/payment/create-order/{bookingId}
        → PaymentController creates Razorpay order (amount in paise)
        → Stores razorpayOrderId in Booking
        → Returns {orderId, keyId, amount} to frontend
        → Razorpay checkout opens in browser

Step 10: After payment
        → POST /api/payment/verify
        → PaymentController verifies Razorpay signature using HMAC
        → Updates booking.paymentStatus = "PAID"
        → EmailService sends confirmation email to tenant
        → Returns success response
```

---

9. OWNER PROPERTY CHANGE REQUEST FLOW

```
Owner adds property
  → POST /api/owner/properties
  → Creates a "change_requests" record with changeType=ADD, status=PENDING
  → DOES NOT directly create property in pgs table yet

Admin reviews in AdminDashboard > Requests
  → GET /api/admin/requests → Shows PENDING requests
  → Admin clicks Approve
  → PUT /api/admin/requests/{id}?action=APPROVE
  → AdminRequestService: reads changeDetails JSON, creates Property in pgs table
  → Or REJECT: simply updates status to REJECTED

Owner is notified (optionally via email)
```

---

10. IMAGE UPLOAD FLOW (Cloudinary)

```
Owner goes to Add Property form
  → Selects image file
  → POST /api/files/upload (multipart/form-data)
  → FileUploadController calls CloudinaryService.upload(file)
  → Cloudinary API returns public image URL (CDN link)
  → URL stored in Property.imageUrl (LONGTEXT column)
  → All users (CLIENT, OWNER, ADMIN) can view image via public CDN URL
```

---

11. AOP LOGGING (Cross-Cutting Concern)

`LoggingAspect.java` uses `@Around` advice on all controllers:

```java
@Around("execution(* com.residentia.controller..*(..))")
public Object logControllerMethods(ProceedingJoinPoint joinPoint) {
    // Logs: class name, method name, HTTP method, URI, user email, execution time
(ms)
    // On success: logs ☑ completion + duration
    // On exception: logs ❌ error details
}
```

This means **every single API call** is automatically logged — no manual logging needed in controllers for basic request/response tracking.

---

12. EXCEPTION HANDLING STRATEGY

```
GlobalExceptionHandler (@RestControllerAdvice)
├── ResourceNotFoundException → 404 NOT FOUND
├── DuplicateResourceException → 409 CONFLICT
├── UnauthorizedException → 401 UNAUTHORIZED
├── MethodArgumentNotValidException → 400 BAD REQUEST (with field errors map)
└── Exception (catch-all) → 500 INTERNAL SERVER ERROR

All errors return structured ErrorResponse:
{
  "status": 404,
  "message": "Property not found with id: 5",
  "error": "ResourceNotFoundException",
  "path": "/api/client/properties/5",
  "timestamp": "2026-03-01T10:30:00"
}
```

---

13. FRONTEND AUTH STATE MANAGEMENT

No Redux / Context API — uses `localStorage`:

```
// On login success (frontAuth.js):
localStorage.setItem("pg_auth", JSON.stringify({
  email: "user@gmail.com",
  role: "CLIENT",
  isLoggedIn: true,
  token: "eyJhbGciOiJIUzUxMiJ9...",
  user: { id: 5, name: "Prajwal", email: "user@gmail.com" }
}));

// On every Axios request (axios.js interceptor):
const auth = JSON.parse(localStorage.getItem("pg_auth") || "{}");
config.headers.Authorization = `Bearer ${auth.token}`;

// ProtectedRoute reads:
const auth = getAuthUser(); // → JSON.parse(localStorage.getItem("pg_auth"))
if (!auth?.isLoggedIn) → redirect
if (auth.role !== requiredRole) → redirect
```

---

14. CORS CONFIGURATION

Backend allows these origins (development):

- `http://localhost:3000`, `http://localhost:5173`, `http://localhost:5174`, `http://localhost:5175`
- Method: `GET, POST, PUT, DELETE, OPTIONS, PATCH`
- `allowCredentials: true`
- Preflight `OPTIONS` requests: `permitAll()`

15. KEY DESIGN DECISIONS — INTERVIEW QUESTIONS

Q: Why JWT and not sessions?

> JWT is stateless. No server-side session storage needed. Backend is deployed on AWS EC2 and scales horizontally — sessions would require sticky sessions or Redis. JWT is self-contained and carries role information.

Q: Why separate Owner and Client registration flows?

> They have different entity tables (`regular_users` vs `pg_owners`) with different fields (businessName, verificationStatus for owners). They have different roles and different API access patterns.

Q: Why does Admin have hardcoded credentials?

> Admin is a single trusted entity, not end-user. The initial admin has hardcoded credentials `admin@residentia.com / Admin@123` for bootstrapping. Secondary admins can be stored in the `admins` table.

Q: How does the Owner permission work for bookings?

> The JWT for OWNER role contains `ownerId` as a custom claim. `JwtAuthenticationFilter` extracts it and sets `request.setAttribute("ownerId", ownerId)`. Controllers read this attribute to scope queries to only that owner's properties. This prevents owner A from viewing owner B's bookings.

Q: Why use AOP for logging?

> AOP separates the logging concern from business logic. Without AOP, every controller method would need repeated `log.info(...)` boilerplate. The `LoggingAspect` intercepts ALL controller calls automatically via pointcut expression.

Q: How is the property listing public but bookings protected?

> In `SecurityConfig.java`:
>
> ```
> .requestMatchers(HttpMethod.GET, "/api/client/properties").permitAll()  // public
> .requestMatchers("/api/client/**").authenticated()  // rest protected
> ```
>
> The order of rules matters — Spring Security matches the first applicable rule.

Q: What is the Change Request flow?

> Owners cannot directly publish PGs. They submit a `change_request` (stored with `changeDetails` as JSON). Admins review and approve/reject. On approval, the actual `Property` record is created/updated. This gives the Admin an approval gate.

Q: What happens on payment?

> 1. Client creates a Razorpay order (`POST /api/payment/create-order/{bookingId}`)

> 2. Backend calls Razorpay API → gets `orderId`
> 3. Frontend opens Razorpay checkout popup
> 4. On payment success, Razorpay sends `paymentId`, `orderId`, `signature` to frontend
> 5. Frontend calls `POST /api/payment/verify` with these three fields
> 6. Backend verifies HMAC-SHA256 signature: `HMAC(orderId + "|" + paymentId, keySecret)`
> 7. If valid → update `paymentStatus = PAID`, send email confirmation

---

16. DEPLOYMENT ARCHITECTURE

```
Internet → AWS EC2 (Spring Boot :8080)
                  ↓
           AWS RDS MySQL (residentia_db)
           ap-south-1 region

Frontend: Runs locally (Vite dev server) or can be deployed to S3/Vercel
Backend API Base: http://13.235.51.122 (from axios.js baseURL)
```

---

17. QUICK INTERVIEW SUMMARY (30-second pitch)

> "Residentia is a three-role PG Finder platform. The backend is Spring Boot with Spring Security using JWT-based stateless authentication. We have three user types — CLIENT, OWNER, and ADMIN — each with separate entity tables and API groups. Frontend is React with React Router, protected routes based on localStorage-stored JWT data. Key flows include: property listing (public), booking creation (client auth), Razorpay payment integration, Cloudinary image storage, and an owner change-request approval flow managed by admin. Backend uses AOP for cross-cutting logging, a global exception handler for standardized error responses, and Spring Data JPA with a MySQL database hosted on AWS RDS."